A Software Architecture for Developing Distributed Games that Teach Coding and Algorithmic Thinking

Nearchos Paspallis
School of Sciences
Univ. of Central Lancashire–Cyprus
Larnaca, Cyprus
npaspallis@uclan.ac.uk

Nicos Kasenides
School of Sciences
Univ. of Central Lancashire–Cyprus
Larnaca, Cyprus
nkasenides@uclan.ac.uk

Andriani Piki
School of Sciences
Univ. of Central Lancashire–Cyprus
Larnaca, Cyprus
apiki@uclan.ac.uk

2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC) | 978-1-6654-8810-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/COMPSAC54236.2022.00023

***Abstract*—This paper presents an architecture for building multiplayer games that aim to teach coding skills and promote algorithmic thinking. The main requirements for the architecture are to enable quick and affordable development and deployment, support commodity client devices, and enable multiplayer, competitive gameplay. By demonstrating an evaluation case study, we show how the proposed architecture achieves these requirements. At its core, it realizes a distributed model extending the client-server paradigm, where multiple players can independently train, then compete in a multiplayer mode using a shared, cloud-based server. While the architecture is validated with a specific maze-themed case study game, we argue that the main principles of this approach can be reused to a wider range of multiplayer, educational games.**

***Index Terms*—software architecture, middleware, computer science education, gamification**

I. Introduction

Coding and algorithmic thinking are increasingly considered core skills [1]. While more people become increasingly active in the modern information-based world, most of them remain to the *consuming* side of computing and only a few delve into the more creative side of *coding* and *algorithmic thinking* [2].

Many initiatives aim to reverse this trend, by inspiring more people to take an interest in coding. For instance, the *Hour of Code*—originally launched in the United States—mobilizes popular personalities from the area of computing and beyond to inspire students to try out coding [3]. Its mission is to "*[...] show that anybody can learn the basics, and to broaden participation in the field of computer science*" [4]. Similarly, *Code Week* in the European Union aims to "*[...] bring coding and digital literacy to everybody in a fun and engaging way*" [5]. Finally, smaller-scale national initiatives aim for the same goal, incorporating various hands-on approaches such as specially themed workshops to attract students to learn coding [6].

Block-based visual languages have been used widely to introduce learners into programming as they minimize the risk of syntax errors while providing visual clues on the relationship of the various programming constructs. Perhaps the most popular such language is Blockly [7], the underlying language behind *Blockly.Games* [8] and the widely popular Scratch platform [9]. Blockly-based approaches have been used in many contexts, with many researchers feeding back positively for its effectiveness in engaging learners [10]–[12]. However, visual programming languages such as *Blockly* and *Scratch* have also been criticized for not *preparing* learners for traditional, script-based programming skills which are in high demand in the industry. Still, proponents of these approaches argue that employing *Blockly*-like learning at first can be a successful approach—as it can ease the learners into the programming concepts—and then prepare them for *script*-like languages by also showing the generated code—*i.e.*, defining an *exit strategy* [13].

Naturally, some projects have aimed to encourage code learning directly using text-based languages. For example, *Code Combat* is "*a game-based computer science program where students type real code and see their characters react in real time*" [14]. *Code Combat* enables players to define code using JavaScript or Python, which are some of the more commercially successful and widely used languages. However, some researchers have argued that this kind of games can be biased—for example, in terms of gender, they appear to be discouraging girls from pursuing coding [15].

Similar to *Code Combat*, an open, multiplayer game named *RoboCode* has been developed and used in traditional problem-based learning with positive feedback [16]. *RoboCode* has also been assessed in supporting the delivery of specialized topics such as Artificial Intelligence [17]. A survey of relevant approaches is presented in a paper by Combefis et al. [18] where they review the main kinds of online platforms to learn programming and more general computer science concepts.

Lastly, it should be pointed out that the last decade has seen many innovative ideas involving the ubiquitous smartphone [19], as well as *physical toys* [20]. These ideas aim at delivering a playful experience to users, while also introducing them to basic concepts of coding and algorithmic thinking.

In this paper, we present a software architecture for developing distributed educational games that aim to attract more students to take an interest in coding. Specifically, our architecture enables puzzle-like games where the players interact with them *indirectly*, by defining code. Using a step-by-step approach, techniques from gamification, and a multiplayer mode, we aspire to enable a breed of games that interest and engage newcomers into the world of coding and algorithmic thinking.

978-1-6654-8810-5/22/$31.00 ©2022 IEEE
DOI 10.1109/COMPSAC54236.2022.00023

In this work, we tackle the following research questions:

- How can we quickly and affordably develop new games that teach coding and algorithmic thinking?
- How can visual programming-based languages be integrated in such games?
- How can such games be extended to enable concurrent, multiplayer mode?
- How can we assess whether the resulting games are effective in promoting the complexity, wealth and value of learning to code?

The focus of the paper is on proposing an architecture which enables the formation of educational games, rather than discussing in detail specific strategies, methods or techniques for enabling learning. It is argued that the proposed methods and tools enable the quick formation of educational games, but a detailed analysis of educational techniques is beyond the scope of this paper.

The rest of this paper is organized as follows: Section II lists the requirements and presents an outline of the targeted game model and design. Then section III describes the main components and the architecture of the proposed approach. The architecture highlights the similarities—and opportunity for reuse—of components across the local and the distributed versions of the implemented games. The architecture is further demonstrated and evaluated in section IV, via its application to a case study that realizes a multiplayer maze-solving game. This is followed by a description of our evaluation results in section V. The paper closes with conclusions and pointers to future work in section VI.

II. Requirements and Design

Learning computer programming requires two ingredients. Firstly, developing a working knowledge of the building blocks of programming, such as *variables*, *conditionals*, *loops*, *functions*, etc. Secondly, developing the ability to analyze problems and specify solutions—*i.e.*, *algorithmic thinking* [2].

To simplify the first part, we build on *Blockly* [21]—a visual programming approach with jigsaw puzzle-like pieces widely known via the *Scratch* programming environment for creating interactive stories, games, and animations [9], [22].

Furthermore, to enable algorithmic thinking, we develop a series of game levels which gradually expose the players to increasingly more complex and algorithmically demanding challenges.

The main goals of the architecture are to enable the following features:

- Learners can play simple games, similar to *Blockly Games* [8], which help them learn programming.
- Users can define code using a graphical programming language, such as *Blockly* [21].
- Users are further motivated by enabling them to compete with each other. Competition can be provided both in terms of *Gamification* [23], *e.g.*, via leaderboards, but also in terms of real-time competition within the game, *e.g.*, via online multiplayer mode.

Additional requirements include the ability to run on commodity hand-held devices such as Android-based smart-phones and tablets, and the ability to work both in stand-alone mode (on device) as well as in multiplayer mode (over the network).

A. Model

The architecture defines its *model* via these *concepts*:

- A *Player* (*i.e.*, *Learner*) is a human who plays the game. Typically, the player controls an in-game character, referred to as *Actor*.
- An *Actor* is a code-controlled character, similar to *Pacman*'s titular protagonist. As mentioned in the requirements, in the proposed architecture players do not control actors directly, *e.g.*, by means of a joystick, but rather indirectly by defining the code that realizes their *logic*. Both physical—*i.e.*, human defined—and game-controlled players are possible. However for the purposes of *proof-of-concept*, only physical players have been implemented so far.
- An *Object* is a game element other than the actors, such as *pickables* and *obstacles*. The former are positive objects which reward the player, while the latter are negative objects such as road-blocks and mines.
- A *Grid* is a model of the terrain, defining the space—and boundaries—in which the actors and the various game objects operate. For example, a grid can be a fully enclosed maze grid.
- The *Game State* describes the state of the game, *i.e.*, of *Players* and other *Game Elements*, at a specific point in time.
- The *Actions* is the set of the possible *moves* or *decisions*, allowed at each round. For example moving, turning, or interacting with other game Objects.
- The *Rules* control the set of actions that individual players can take in each round, as well as the outcome of each action given the game state.
- A *Challenge* is essentially an instance of a game in runtime. A game can include multiple concurrent *Players* with their corresponding *Actors*, who compete at the same time and space. The players can either challenge each other directly (*e.g.*, they can set obstacles for competitors, such as time-bombs), or indirectly (*e.g.*, by simply competing on a score basis, such as racing to be the first to find the exit in a maze or collect the most points).

B. Design

The game is the combination of the above defined concepts. To better illustrate this, consider an implementation of a simple maze-solving game, *i.e.*, one where the player must define the algorithm to guide an actor to the exit of the maze. For reference, consider the code and runtime view shown in figures 1 and 2 respectively.

For this game, the concepts are mapped as follows:

- Assume a singleplayer game for now. The *Player* is the human using the game and writing the code.

- The *Actor* is the avatar controlled by the user's code, and which works its way out of the maze. In the maze of figure 2, this is depicted by a simple *triangle* with the label "You" on top.
- The *Objects* are various elements of the game. For example, the maze challenge could have a more challenging mode, employing different *pickables*—*e.g.*, positive elements such as fruit which increase the score or health—and *obstacles*—*e.g.*, negative elements such as yellow warning triangles which decrease health. For the purposes of this simplified screenshot—*i.e.*, figure 2—we assume that there are no objects used.
- The *Grid* is the maze and the outer border—shown as thin white lines in figure 2.
- The *Game State* is simply the location and direction of the *Actor*. If other game *Objects* were used, or multiplayer mode was enabled, then the state would include those too.
- The *Rules* map all possible actions to their outcomes—*i.e.*, how the *Game State* is updated. An example in this case is that the *Actor* cannot cross over walls. The possible actions for an *Actor* are: *move forward*, *turn 90° clockwise*, and *turn 90° counter-clockwise* or *Do Nothing*–cf. the Blockly commands in figure 1.

C. Gameplay

The game first guides the player to define their code, and then apply it to the game. Once submitted, the code is applied and the player observes the progress and effectiveness of their code. The code is defined using standard Blockly blocks, such as *while*, *if*, get and set *variables*, etc. The starting points are set in two special functions:

1) The `Initialize()` function provides the opportunity to set the variables before the actual round-based play commences. The player can leave this function empty if they do not need to initialize anything.
2) The `Run()` function defines the logic to be followed by the Actor. This code is called repeatedly in set intervals, *e.g.*, every one second. Each execution must pick exactly one Action, *i.e.*, the next move. If this is a valid move, it is applied, otherwise it is ignored—effectively the Player loses their turn for the round. Furthermore, the Player is only given a set amount of time to execute their code, after which the game is terminated and the Player loses their turn. This protects from infinite loops and from excessively demanding algorithms. For instance, a bug in the Player's code could cause the program to loop infinitely, so unless it is watched and stopped at a set time, it could freeze the execution of the game for all players. Similarly, excessively demanding algorithms could impact the interactivity of the game. By setting a watch and terminating its execution after a set time, we mitigate both these risks.

This is illustrated in figure 1. The Player defines the *left wall following* algorithm, which can solve a non-looping maze, using the relevant blocks. The algorithm is successfully applied to a simple maze, and demonstrates how the Actor is controlled by it.

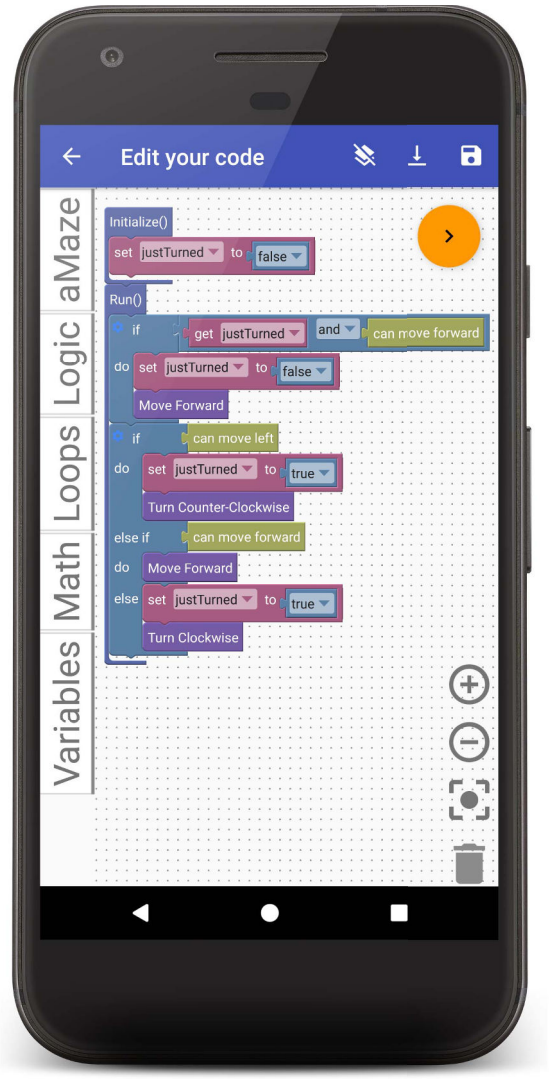

Fig. 1. Screenshot illustrating the gameplay. The coding of the *left wall follower* algorithm is defined using the `initialize` and `run` functions.

III. Software Components and Architecture

A. Core Components

Computer games can be quite sophisticated, with detailed graphics and complex gameplay rules. multiplayer, distributed games add further complexity. In this paper, we focus on turn-based games which aim to introduce or improve coding skills. As such, the focus is mostly on cultivating algorithmic thinking rather than on advanced graphics.

The main components used in this architecture are:

- *Game API*: This *Application Programming Interface* defines the parts of the game which are accessible to the *Player*. This is primarily the set of *commands* available to the player during the definition of the algorithm. For

Fig. 2. Screenshot illustrating the gameplay. The *left wall follower* algorithm is tested in a maze, and used to successfully guide the Actor to the exit (green square in the top-right corner).

example, in the maze game the commands are *move forward*, *turn clockwise*, etc. The actions received via the API are used to update the game *State*, while abiding by the *Rules*.

- *Blockly Editor*: Used by the player to edit their code in a drag-and-drop fashion towards defining their algorithm. The available programming structures include standard Blockly elements, such as *loops* and *conditionals*, as well as the game-specific elements—such as "Move Forward"—defined in the *Game API*. Once ready, the set code is compiled—typically to JavaScript, even though other languages such as Python are also supported—and then passed on to the *Game Runtime* to be used to control the player's avatar.
- *Game Runtime*: The main role of the runtime component is to *interpret* the player's code and apply the resulting actions via the *Game API*, thus updating the *game state*, and effectively advancing the game progress. This for instance checks how the avatar interacts with the various game elements, updates the score and the user health, and checks for conditions which could affect the game progress—*e.g.*, running out of time, or depleted health.

B. Local and Distributed Architectures

These components are combined to form two architectures, a *local* and a *distributed* one. The former serves for playing singleplayer, offline games—*i.e.*, locally on the Player's device. The latter is used to realize multiplayer games—*i.e.*, over the network.

1) Local Architecture: The local architecture is illustrated in figure 3. The developer first needs to develop blocks for the *Custom Commands*. These blocks are created using the *Blockly Developer Tools* [7].

At runtime, when the players finish defining their code, they *submit* it. This action triggers a set of tests, which can produce any number of *compilation* or *logic* errors, and warnings. If any error is raised, the step fails and the user is asked to continue editing the code. If no errors are raised, the compiler generates the equivalent JavaScript code.

The generated code is used by the *Game Runtime* to decide the next action, in each round. Typically, the rounds progress at a relatively slow pace—*e.g.*, every one second—to allow the player to *observe* the progress of the game and possibly identify corrective actions for their code. In stand-alone mode, special game widgets allow the user to *manually* pace the execution of the game, *e.g.*, by pressing the "Next" button, or by setting the "Interval" between automatic invocations of the actions, as shown in in figure 2. These features are explicitly designed to enable the Player to have as much feedback as possible regarding the execution of their code, allow them to be in control of the execution, and thus assist them in their learning.

As mentioned in section II-C, the first time the code is executed, the *Initialize* function allows the Player to define variables and assign initial values. From that point on-wards, the *Run* function is called periodically to decide the user action in each round.

This form of *interpretation-based* execution is by default *stateless*. However, in many cases the users need to define code which *persists* across subsequent rounds of execution. For example, the variable *justTurned* in the left-hand wall follower algorithm—shown in figure 1—must be persisted across invocations for the algorithm to work.

State persistence for the used variables is achieved by amending the JavaScript code and by adding provisions in the *Game Runtime*. More details about this are provided later in the discussion of the implementation in section IV.

Finally, in each invocation round, the *Game Runtime* computes the player's choice in the form of an action. This is applied to the game via the *Game API*. Effectively, the game updates its state by implementing the action, while of course

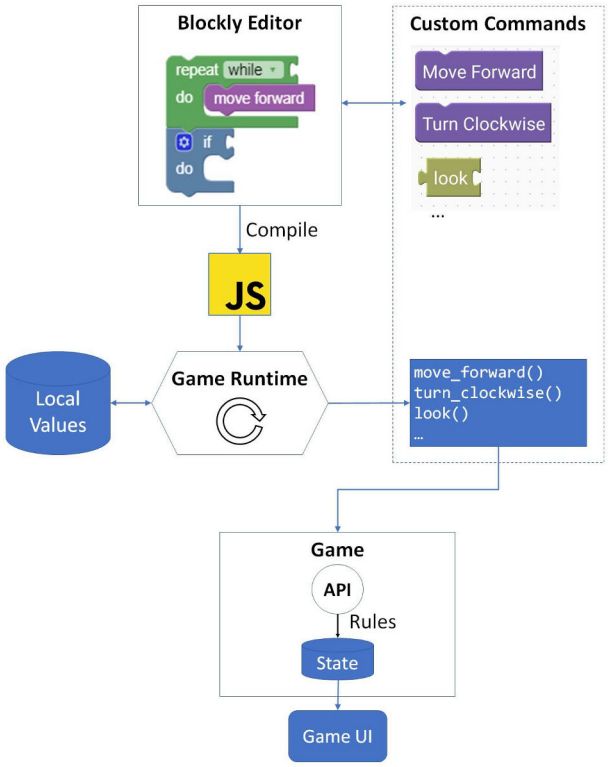

Fig. 3. The local version of the proposed architecture.

ensuring the *Rules* are adhered to. Based on this, the *State* is used to update the *Game User Interface* (*Game UI*).

2) Distributed Architecture: The local architecture is extended to support multiplayer gameplay by splitting it to a client-side containing the *Blockly Editor* and the *Game UI*, and to a server-side containing the *Game Runtime* and the shared state. This architecture is illustrated in figure 4.

Individual players define their code using the *Blockly Editor* on their own client device. The code can be *tested* locally—using the local architecture described earlier—and subsequently it is checked for compilation and logic errors or warnings on site.

When ready, the compiled JavaScript code is *uploaded* to the centralized server via the Network. Similarly, the individual clients receive updates concerning the game state also via the Network, which in return enables them to provide a continuously updated, local version of the *Game UI*.

On the server-side, the uploads are received and a queue is maintained with the latest code from each participating client. If the game has a limit for the number of concurrent users, the queue is also used as the *staging area*—where the first *N* players are active, and the remaining are queued to join first-come-first-served when an active player is deactivated.

For the active players, the *Game Runtime* executes the corresponding JavaScript code with the selected frequency. Similar to the local architecture, the used variables are persisted for as long as the corresponding player's code is active. The selected *actions* are used to update the shared game *State* via its API.

Updates to the state are communicated back to the players—and possibly to other, passive listeners too—so they can update their *Game UI*. This can be realized either via polling, or an event distribution mechanism. Note that while the round-trip time—*i.e.*, latency—is a crucial metric for multiplayer action games, in this case this is not so critical as the players do not control the game in real-time. The players need only receive the update events in an ordered, near-time—rather than real-time—manner, *i.e.*, within a second of the update happening on the server.

3) The Network Interface: To facilitate the interfacing of clients with the server, we define an API which is implemented as a REST-ful architecture [24]. This interface provides both core game, and management functionality.

Specifically, the core fame functionality available via the API includes the ability to *upload code*, to receive *state updates*, and *request other metadata* such as the *score*. On the other hand, the management features enable a client to *list* the available *challenges*, request to *join* one, and also to *leave* one which it had previously joined.

Furthermore, a set of administrative functions allow for the back-end managers to define *new challenges*, and *manage* existing ones—*i.e.*, to set their parameters, activate and deactivate them, etc.

IV. Case Study

The proposed architecture was used to implement *aMazeChallenge*, a working prototype serving as a *proof-of-concept*, as well as for a preliminary evaluation—which is discussed in section V.

We describe the main considerations regarding the implementation of the architecture, and specifically aspects that relate to:

- Project organization to maximize reusability.
- Embedding of the *Blockly* library.
- Testing for compilation errors, logic errors, and warnings.
- Compilation to JavaScript.
- Interpreting the JavaScript code with *Mozilla Rhino*—a 3rd-party library [25]—while preserving the state.
- Implementation of the client-side for Android/Java.
- Implementation of the server-side for App Engine/Java.

A. Project structure

One of the main aims of the implementation was to organize the code in projects, in order to facilitate reusability. This has been assisted by the fact that all code was implemented in Java [26], [27], thus enabling reusability. Specifically, the client-side was implemented using *Android/Java* and the server-side was implemented using *App Engine/Java*. Furthermore, we utilized Java-based libraries such as *Mozilla Rhino* and the cross-platform *Protocol Buffers*, which further facilitated reusability.

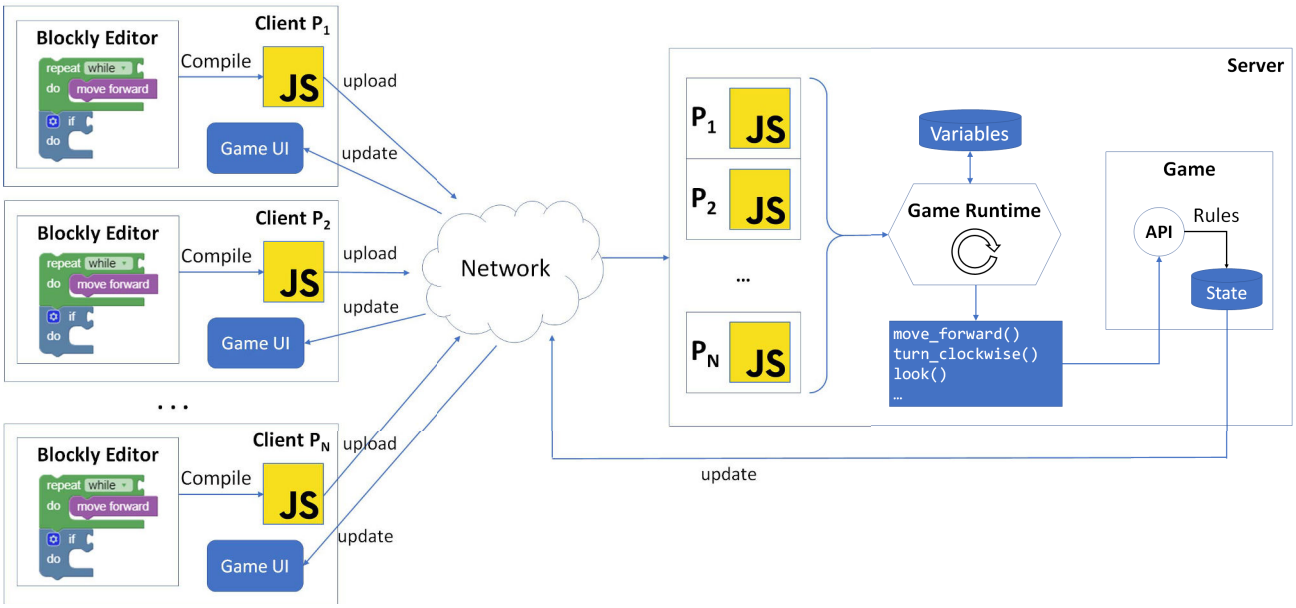

Fig. 4. The distributed, multiplayer version of the proposed architecture. The client-side hosts the *Blockly Editor* and the *Game UI* while the server-side provides a centralized *Game Runtime* with a single game state.

The project code is organized in smaller sub-projects: the *Client* (Android) sub-project, the *Server* (App Engine) sub-project, and a *Common* sub-project for functionality required by both the client and the server sides. For instance, this includes the code which is used to *interpret* the JavaScript and to realize the *Game* with model, state, and other data structures—see figures 3 and 4 respectively.

The shared project also includes the messages exchanged between the client and the server—in their Java form. The complete *aMazeChallenge* project source code—including the *client* and the *server* components—is available online under an LGPL open source license [28].

B. Embedding Blockly in the Client

Players utilize Blockly [13], [21], a visual block language developed by Google, to write their maze-solving code. Google has developed an Android library for Blockly, which allows the language to be easily integrated into native Android apps. The package includes `AbstractBlocklyActivity`, which developers must implement in their own Android activities to embed Blockly workspaces into apps. In our client this is implemented by `BlocklyActivity`, which enables:

- Opening and observing a Blockly *toolbox*, which contains the default (code-logic) and the custom (game-specific) blocks, categorized in groups.
- Inserting, modifying, and removing blocks from a Blockly workspace.
- Converting code from a Blockly workspace into XML format.
- Loading pre-existing code files into the workspace.

C. Testing for Compilation and Logic Errors, and Warnings

While the `Initialize` function is optional and can be omitted, the `Run` function is mandatory and needs to be included for the code to be valid. This and many other checks are built into the architecture. When the *Player* finishes with programming, the code can be explicitly compiled by pressing a button, or, have it automatically triggered upon exiting the code editor page. At that point, the custom tests are first checked before deploying the code.

Despite the relatively strict typing of Blockly, mistakes can still occur in *Player*-defined code. Before compiling to JavaScript, the client conducts several static code checks to verify that the code does not include any obvious errors. At this intermediary stage, the blocks are translated into XML format and several checks occur, such as checking for: *empty or missing Initialize or Run functions*, *empty compound statements* (such as loops without a body), *empty conditional statements* (such as if-statements without a condition), *function-in-function statements* (inserting functions inside functions is not permitted in this case), *inifinite loops* and *Run function re-definitions*.

Possible code mistakes are categorized as either *Warnings* or *Errors*. Just like in most IDEs, *Players* may ignore warnings, but are required to fix errors before the code can be successfully compiled and deployed.

D. Transformation to JavaScript

Upon passing the inspection, the code is compiled to JavaScript by the built-in Blockly compiler. Nevertheless, the resulting code is not ready to be executed yet. First, a way is needed to pass data from the host, Java-based environment to the interpreted, JavaScript-based one. Second, extra code

is needed for picking the values of defined-variables *after* each round of execution, and for initializing said variables *before* each subsequent round of execution. Finally, special provisions are needed for realizing functions which are defined in the *Game API* and are called in the Blockly-defined code. This is achieved by a *processing stage* which injects additional JavaScript code to the generated one. At that point, the code is ready to be executed, either locally or remotely on the backend.

E. Interpreting the JavaScript code

At the core of the project is the user-defined code and its *interpretation* for realising the gameplay. For this purpose, we use a third-party library: Mozilla Rhino [25]. This library enables invocation of JavaScript code inside a Java program, by defining the entry point—*e.g.*, a function name. Furthermore, it enables interaction with it, as it can also access Java values passed as arguments, and can return values as well.

In our implementation, the `Initialize` function is invoked first, followed by the periodic invocation of `Run`. The latter function must return a value: the decided *Custom Command*. The code is provided with access to the relevant *Game API* via an argument passed during execution.

Lastly, the amended JavaScript code has a special preamble—discussed earlier—which initializes all variables with the values stored in a persistence hashtable serving as a cache. At the end of each round, the updated variable values are again saved in the hashtable. This enables persistence of JavaScript-defined variables across the various execution cycles.

F. Android Client

An Android app is developed which can be used to run stand-alone, offline games, as well as serve as the client-side for multiplayer, online games. The app provides a sequence of actions which can be used to engage the players: Learn by reading help files, personalize their avatars, edit and try out code for certain problems, and compete online. For advanced users, the client even offers a maze designer which enables the players to define their own challenge, with a custom maze and game elements.

With the use of *empty checkboxes* and *ticks*, the menu doubles as a gamification mechanism encouraging the player to progressively advance to the next steps and complete more challenges. Additional game elements include a *Guide* page with learning material and screenshots, the *Code Editor* with general and game-specific blocks, the *Training* page to locally test the code, and the *Online* page to compete with others. These functionalities are illustrated in figure 5.

G. App Engine Server

The server-side realizes the backend for the multiplayer game. For the prototype implementation, we opted for App Engine/Java [29]. This choice offers relatively seamless scalability—even though a detailed discussion of this feature is beyond the scope of this paper. Furthermore, it offers many of the advantages of cloud computing, such as low entrance cost, high availability, etc.

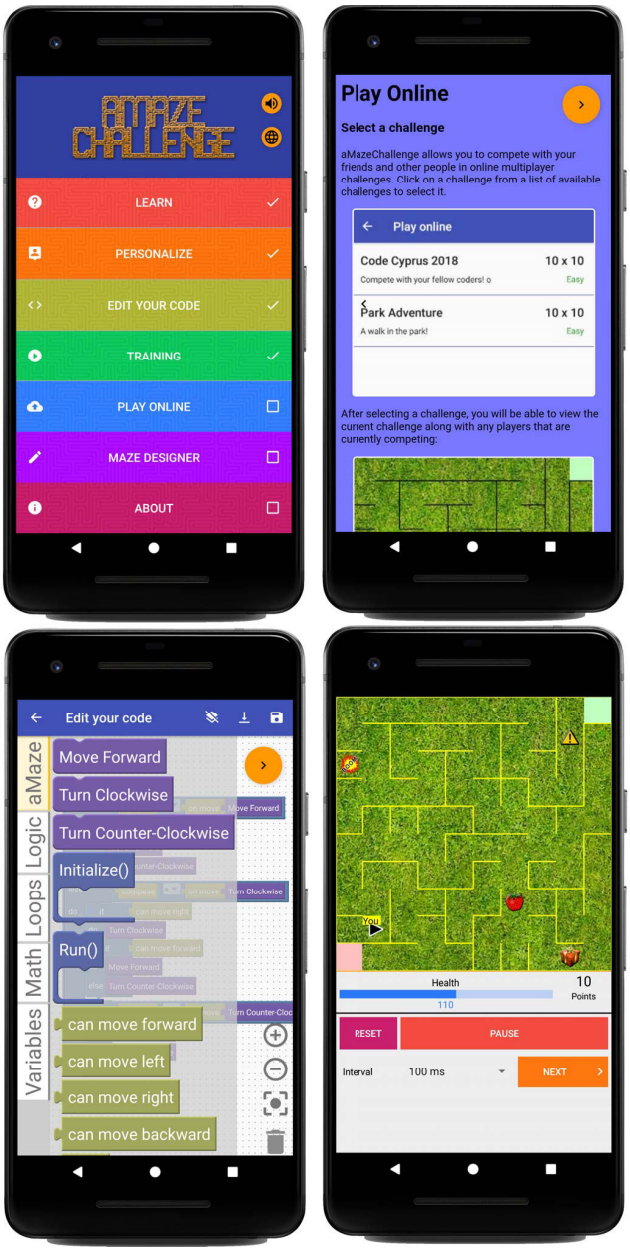

Fig. 5. Screenshots of the Android-based client. The main menu (top left) encourages the learners to complete the available game steps. An in-app guide (top right) helps learners understand the basics of coding. The Blockly-based editor (bottom left) provides a familiar, visual programming environment. The game view (bottom right) shows the grid and visualizes the execution of their code.

The server is used to provide two main functionalities: a public API which is used by the clients, and a private set of administrative web pages which is used by the system owner to monitor and manage the deployed challenges.

The API is implemented as a set of REST-ful functions [30]. All of them assume that the client initiates a request, and the server returns the corresponding resource encoded as a bytestream using Protocol Buffers—*i.e.*, there is no push-notification functionality. This architecture enables various implementations for clients—such as *Web Apps* and *iOS Apps*—even though for the purposes of the *proof-of-concept*, we provide just an Android-based implementation.

For data persistency the server utilizes the Google Datastore, a "highly scalable NoSQL database" [31]. This is used to keep data of the active *challenges* as well as of the *runtime* data of the active sessions—*i.e.*, the *Game State*.

V. Evaluation

The evaluation of the architecture is twofold: First, we discuss the contributions of the architecture in the context of related work. Next, we assess the architecture based on the detailed case study presented earlier.

A. Related Work

The Software Engineering Institute (SEI) defines *Software Architecture* as "[the artefact which] *represents the design decisions related to overall system structure and behavior*" [32]. Adding to the definition, *Clements et al.* argue that "*effectively documenting an architecture is as important as crafting it, because if the architecture is not understood (or worse, misunderstood) it cannot meet its goals*" [33].

Developing games for facilitating programming education is a popular field, evident by the large number of papers published on the topic. In [34], the authors evaluate 49 serious games in terms of likability, accessibility, learning effect and engagement. They conclude by identifying a number of open problems in the literature. Very importantly, the authors identify a lack of multiplayer games, identifying "*a need for further research on the learning benefits of competitive and collaborative serious games for programming.*"

In another extensive literature review, Laine and Lindberg [35] review 11 studies on game motivators, and another 41 on design principles. One of the main game motivators identified is *competition*, which justifies the choice for a software architecture which supports multiplayer, online games. Specifically, the authors argue that such multiplayer games enable competition which is "*a motivator that arises when the player has to compete with other players to reach a goal.*"

More relevant to developing educational games, Hu proposes a common software architecture [36]. In this architecture, the developers need to analyse their game by identifying four core concepts: Subject, Activities, Task, and Feedback. The author argues that cross-platform reusability is achieved by means of adopting a high-level abstraction for educational games, with no dependency on underlying algorithms and data structures, and thus no dependency on game engines. Similar to our approach, the architecture is validated via a case-study game, titled "*Breath: A Sim-Eduventure*".

Another approach to building games for programming education is described by Xinogalos and Tryfou [37]. In their paper, they assess whether the *Greenfoot* tool [38] can be used "*for an introduction to (serious) game development*". Their results build on their own experience with building a serious game titled "*Game of Code: Lost in Javaland*" using Greenfoot. Their game aims at introducing primary school students to core concepts of Object-Oriented Programming (OOP). A survey-based evaluation indicated that their approach is indeed a viable method for developing serious educational games.

B. Case Study-based Evaluation

The Case Study presented in section IV showed that the architecture is applicable and realistic. Alpha and beta testing demonstrated that the implementation of both the local and the online/distributed architectures exhibit adequate performance.

In this section, we further argue that the proposed architecture promotes reusable components. In this way, it minimizes the need for additional code both when developing the local and distributed versions of the same game, but also for when developing separate games.

1) Reusability of code across local (single) and distributed (multiplayer) versions: When the architecture is used to develop the local and distributed versions of the same game, the following components are reused:

- The *Blockly Editor*, including the *Custom Commands* for the game.
- The *Game Runtime*, including the cache which is used for storing local values.
- The *Game API* which realises the game rules and updates the state. This also includes the game-specific commands for handling the *Custom Commands* defined in Blockly.
- The *Game UI* which is driven by the state and draws the game graphical interface, and plays the audible effects.

These components can be reused verbatim, with the following exception: The cache for storing local values must be amended for the distributed version, to facilitate storing different copies of the same variable (once per player). The *Blockly Editor* behaves the same way and generates the same output, for both the single and the multiplayer versions. The *Game Runtime* interprets the code to update the *Game State*. The same component used for the singleplayer version can be reused for the multiplayer one, with simply adding wrapper code to apply the JavaScript code for each player, sequentially. The *Game API* is applied verbatim to invocations, one for each player, and updates the *State*. Finally, the *Game UI* is reused as is, because it is designed to be able to deal with any values in the game *State*, including those of a multiplayer game (*e.g.*, two players residing in the same position on the grid).

2) Reusability of code across different games: To assist in the comparison, consider a scenario where a separate educational game were to be developed. For example, a Pacman-like game where the Player controls Pacman (the Avatar) and where the goal is to cover the full grid (i.e., "eat" all the dots)

while avoiding the enemies (*e.g.*, the four "ghosts": Blinky, Pinky, Inky and Clyde).

In this scenario, the developers would first be required to define the game's *Custom Commands*. These would complement the standard Blockly commands for defining variables, implementing conditionals and loops, etc., with the game specific functionality such as "move forward" or "look ahead", etc. Special code would also be required to realise the corresponding command interfaces, so they could be called by the *Game Runtime* during the interpretation of the generated JavaScript code. Finally, the developers should also develop a custom *State* object abstracting the relevant values for the game, such as the positions of Pacman and the ghosts, the state of the grid, etc., and of course realise the game-specific *Game UI*.

The following could be reused verbatim from another game, such as the one presented in the Case Study:

- The *Blockly Editor*.
- The *Game Runtime*, including the cache which is used for storing local values.

The *Blockly Editor* can be reused as is, since it shares the common functionality for realizing variables, conditionals, loops, etc. The visualization of the custom values and commands are driven by the underlying model which must be defined by the developer. The *Game Runtime* can also be reused as is, including the cache. Since the purpose of this component is to interpret the generated JavaScript code, it is agnostic to the game-specific elements. The interface which corresponds to the custom commands is treated in a uniform way from the runtime. Additionally, the cache is fully reusable, as it simply realizes a typical *key-value map*.

C. Discussion

A preliminary user-based evaluation of the architecture was realized by developing and releasing the game discussed in the Case Study section. A limited-scale, student-based evaluation revealed that "*the participants showed that [the app] made them feel more confident in their ability to program*" [39]. Additionally, feedback from the students indicated that one of the most intriguing aspects of the game was the multiplayer mode, which is consistent with the qualities of motivating serious games, discussed earlier in section V-A and supported by the literature.

From a purely educational perspective, games developed with the proposed architecture exhibit some important features. First, the players receive error and warning messages which trigger some reflection on their approach from the early design phase. Second, the learners observe and control the execution of the game in a step-by-step fashion, which enables them to identify corrective actions for their code. For instance they can be warned of empty function bodies, or prompted to fix infinite loops. Third, the multiplayer mode has the advantage of motivating engagement. At the same time, the fact that the learners can upload their code, and observe its performance, then make corrections and retry, provides a valuable opportunity for goal-driven learning [40].

Traditional introductory graphical languages like *Blockly* and *Scratch*, have the benefit of a more universal approach which offers more possibilities for creativity. However, we argue that the proposed architecture, which builds on *Blockly*, has the advantage of being explicitly designed to allow coding on a mobile device, and also to natively support multiplayer games which can boost engagement [41].

On the other hand, visual programming languages, such as *Blockly* and *Scratch*, are sometimes criticised that they do not *prepare* the learners for traditional, script-based programming which is demanded in the industry. Proponents of visual programming however, argue that successful approaches can include *Blockly*-like learning at first—to ease the learners into the programming concepts—and then prepare them for *script*-like languages by also showing the generated code, *i.e.*, defining an *exit strategy* [13].

VI. Conclusions and Future Work

In this paper we presented a software architecture for developing distributed games that aim to teach coding. The evaluation of the architecture showed that it provides a high degree of reusability, which simplifies the creation of novel programming-themed serious games.

The evaluation of the architecture through a Case Study, demonstrates that the research questions have been answered. The first research question concerned on how to quickly and affordably develop new games that teach coding and algorithmic thinking. We argue that the proposed architecture enables quick and affordable development of games as it promotes reusability, and also as it utilizes relevant, open-source components. One of the main components which can be reused is the Blocks editor [7], which can be easily extended and embedded in the flow of the game, answering the second research question. In section V-B1 we argued that reusability does not only extend across different games, but also between local (singleplayer) and distributed (multiplayer) versions of a game. This addresses the third research question, *i.e.*, how the games can be extended to enable multiplayer gameplay. On the other hand, while some preliminary results were collected with a small-scale, student-based evaluation, a more thorough study is needed to fully answer the fourth research question, *i.e.*, assess whether the produced games are effective.

As future work, we aim to enhance the architecture as follows. First, develop a cross-platform, Web-based client to cover most modern mobile smartphones. This should increase adoption as many mobile users do not wish to *commit* to an app straight away, by installing it, and other learners who may not own an Android device. Second, extend the API so that the learners can also engage in blended learning experiences (*e.g.*, program a physical robotic rover and navigate a real terrain to complete a task). Third, we will further evaluate the effectiveness of the approach via additional surveys and focus groups at primary, secondary, and higher education. We will also assess whether advanced users would prefer to have a text-based console to directly enter source code, *e.g.*, in JavaScript. Finally, we will work on developing and evaluating

a pedagogical framework, which leverages the benefits of the proposed architecture.

In conclusion, the proposed software architecture provides a novel approach for building multiplayer, cloud-backed games which can help engage learners to take an interest in computer science in general, and programming and algorithmic thinking in particular. It is argued that such serious games are in demand, as they can be quite effective in motivating learners to take an interest in coding [6] as well as in serving directly as educational artefacts.

REFERENCES

[1] B. Christian and T. Griffiths, *Algorithms to Live By: The Computer Science of Human Decisions*. Henry Holt and Co., 2016.

[2] G. Futschek, "Algorithmic thinking: The key for understanding computer science," in *Proceedings of the 2006 International Conference on Informatics in Secondary Schools - Evolution and Perspectives: The Bridge between Using and Understanding Computers*. Berlin, Heidelberg: Springer-Verlag, 2006, p. 159–168. [Online]. Available: https://doi.org/10.1007/11915355_15

[3] C. Wilson, "Hour of code—a record year for computer science," *ACM Inroads*, vol. 6, no. 1, p. 22, Feb. 2015. [Online]. Available: https://doi.org/10.1145/2723168

[4] (2021) Hour of code: Celebrate computer science everywhere! [Online]. Available: https://hourofcode.com

[5] J. Moreno-León and G. Robles, "The europe code week (codeeu) initiative shaping the skills of future engineers," in *2015 IEEE Global Engineering Education Conference (EDUCON)*, 2015, pp. 561–566.

[6] N. Paspallis, I. Polycarpou, P. Andreou, J. Antoniou, P. Kaimakis, M. Raspopoulos, and M. Terzi, "An experience report on the effectiveness of five themed workshops at inspiring high school students to learn coding," in *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*, ser. ITiCSE 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 105–110. [Online]. Available: https://doi.org/10.1145/3197091.3197093

[7] (2021) Blockly. [Online]. Available: https://developers.google.com/blockly

[8] (2021) Blockly games. [Online]. Available: https://blockly.games/about

[9] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, A. Millner, E. Rosenbaum, J. Silver, B. Silverman, and Y. Kafai, "Scratch: Programming for all," *Commun. ACM*, vol. 52, no. 11, p. 60–67, Nov. 2009. [Online]. Available: https://doi.org/10.1145/1592761.1592779

[10] J. Trower and J. Gray, "Blockly language creation and applications: Visual programming for media computation and bluetooth robotics control," in *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 5. [Online]. Available: https://doi.org/10.1145/2676723.2691871

[11] T. Glushkova, "Application of block programming and game-based learning to enhance interest in computer science," *Journal of innovations and sustainability*, vol. 2, pp. 21–32, 03 2016.

[12] A. Baratè, A. Formica, L. A. Ludovico, and D. Malchiodi, "Fostering computational thinking in secondary school through music - an educational experience based on google blockly," in *Proceedings of the 9th International Conference on Computer Supported Education - Volume 2: CSEDU,*, INSTICC. SciTePress, 2017, pp. 117–124.

[13] N. Fraser, "Ten things we've learned from blockly," in *IEEE Blocks and Beyond Workshop (Blocks and Beyond)*, Atlanta, GA, USA, 2015, pp. 49–50, doi:10.1109/BLOCKS.2015.7369000.

[14] (2021) Code combat. [Online]. Available: https://codecombat.com

[15] Y. Yücel and K. Rızvanoğlu, "Battling gender stereotypes: A user study of a code-learning game, "code combat," with middle school children," *Computers in Human Behavior*, vol. 99, pp. 352–365, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0747563219302109

[16] J. O'Kelly and J. P. Gibson, "Robocode & problem-based learning: A non-prescriptive approach to teaching programming," *SIGCSE Bull.*, vol. 38, no. 3, p. 217–221, Jun. 2006. [Online]. Available: https://doi.org/10.1145/1140123.1140182

[17] K. Hartness, "Robocode: Using games to teach artificial intelligence," in *Journal of Computing Sciences in Colleges*, vol. 19, 2004, pp. 287–291.

[18] S. Combéfis, G. Beresnevičius, and V. Dagienė, "Learning programming through games and contests: overview, characterisation and discussion," in *Olympiads in Informatics*, vol. 10, 2016, pp. 39–60.

[19] D. Crawley. (2014, Jun.) 12 games that teach kids to code — and are even fun, too. [Online]. Available: https://venturebeat.com/2014/06/03/12-games-that-teach-kids-to-code

[20] G. Schmidt. (2017, Dec.) The best toys that teach kids how to code. [Online]. Available: https://www.nytimes.com/2017/12/19/smarter-living/kids-toys-for-coding.html

[21] E. Pasternak, R. Fenichel, and A. N. Marshall, "Tips for creating a block language with blockly," in *2017 IEEE Blocks and Beyond Workshop (B B)*, 2017, pp. 21–24.

[22] (2021) Scratch - imagine, program, share - mit. [Online]. Available: https://scratch.mit.edu

[23] S. Deterding and S. P. Walz, Eds., *The Gameful World: Approaches, Issues, Applications*. MIT Press, 2015.

[24] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[25] (2021) Mozilla rhino - an open-source implementation of javascript written entirely in java. [Online]. Available: https://github.com/mozilla/rhino

[26] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, and D. Smith, *The Java ™Language Specification, Java SE 13 Edition*, 3rd ed. Addison Wesley, 2019. [Online]. Available: https://docs.oracle.com/javase/specs/jls/se13/jls13.pdf

[27] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and D. Smith, *The Java ™Virtual Machine Specification, Java SE 13 Edition*, 3rd ed. Addison Wesley, 2019. [Online]. Available: https://docs.oracle.com/javase/specs/jvms/se13/jvms13.pdf

[28] (2021) amazechallenge - a game-like, distributed application for learning to code by playing. [Online]. Available: https://github.com/nearchos/aMazeChallenge

[29] *Java on Google App Engine*, 2021. [Online]. Available: https://cloud.google.com/appengine/docs/java

[30] L. Richardson and S. Ruby, *RESTful web services*. O'Reilly Media, Inc., 2017.

[31] (2021) Google cloud datastore. [Online]. Available: https://cloud.google.com/datastore

[32] (2021) Software architecture at software engineering institute, carnegie mellon university (cmu). [Online]. Available: https://www.sei.cmu.edu/our-work/software-architecture

[33] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford, *Documenting Software Architectures: Views and Beyond*, 2nd ed. Addison-Wesley Professional, 2010.

[34] M. A. Miljanovic and J. S. Bradbury, "A review of serious games for programming," in *Serious Games*. Cham: Springer International Publishing, Nov. 2018, pp. 204–216.

[35] T. H. Laine and R. S. N. Lindberg, "Designing engaging games for education: A systematic literature review on game motivators and design principles," *IEEE Transactions on Learning Technologies*, vol. 13, no. 4, pp. 804–821, 2020.

[36] W. Hu, "A common software architecture for educational games," in *Entertainment for Education. Digital Techniques and Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 405–416.

[37] S. Xinogalos and M. M. Tryfou, "Using greenfoot as a tool for serious games programming education and development," *International Journal of Serious Games*, vol. 8, no. 2, p. 67–86, Jun. 2021. [Online]. Available: https://journal.seriousgamessociety.org/index.php/IJSG/article/view/425

[38] M. Kölling, "The greenfoot programming environment," *ACM Trans. Comput. Educ.*, vol. 10, no. 4, Nov. 2010. [Online]. Available: https://doi.org/10.1145/1868358.1868361

[39] N. Kasenides and N. Paspallis, "amazechallenge: An interactive multiplayer game for learning to code," in *29th International Conference on Information Systems Development (ISD2021)*. Valencia, Spain: Association for Information Systems, Sep. 2021. [Online]. Available: https://aisel.aisnet.org/isd2014/proceedings2021/methodologies/1/

[40] R. Ashwin and D. Leake, Eds., *Goal-driven learning*. MIT Press, 1995.

[41] N. Kasenides and N. Paspallis, "A systematic mapping study of MMOG backend architectures," *Information*, vol. 10, no. 9, 2019. [Online]. Available: https://www.mdpi.com/2078-2489/10/9/264